

Problem 4, Cell 5: certified boundary, and the sizes $k = 24, 30$

Official statement. For integers a and $m \geq 1$ write $a \pmod{m} = \{a + mt : t \in \mathbb{Z}\}$. A finite family of classes $a_1 \pmod{m_1}, \dots, a_k \pmod{m_k}$ is pairwise disjoint if $a_i \pmod{m_i} \cap a_j \pmod{m_j} = \emptyset$ whenever $i < j$. Nothing is assumed about the moduli beyond $m_i \geq 1$: they may repeat, and the residues a_i may repeat as well. By the Chinese remainder theorem,

$$a_i \pmod{m_i} \cap a_j \pmod{m_j} \neq \emptyset \iff \gcd(m_i, m_j) \mid a_i - a_j. \quad (*)$$

Question. Let $k \geq 2$ and let $a_1 \pmod{m_1}, \dots, a_k \pmod{m_k}$ be pairwise disjoint. Must there be a pair $i < j$ with $\gcd(m_i, m_j) \geq k$?

Cell 5 (The certified boundary, 8 points). The main certification cell: an initial range of sizes as long as you can make it, plus two isolated sizes further out. Determine the largest k for which you can certify the statement for all sizes up to and including k , and certify it. The range you claim must be contiguous, and if your argument at a given size assumes that all smaller sizes have already been settled you must say so. Then decide the two isolated sizes $k = 24$ and $k = 30$: show that neither is the least size at which the statement can fail.

For this cell the exhaustiveness certificate of the hand-in rules is not enough on its own. Add: (i) *a demonstration that your method is not vacuous.* Construct a case in which an admissible configuration genuinely exists, run your machinery on it unmodified, and show it returns that configuration. A method that reports “nothing survives” at every size it is pointed at is indistinguishable from a method with a bug, and will be graded as one; (ii) *for every object your pruning leaves undecided, at every k you claim* — not a sample — an individual decision, plus the smallest part of that object which already forces the decision, plus a proof that no smaller part does; (iii) *the exact list*, not merely the count, of what survives at each k ; (iv) *every pruning rule you use beyond those you have proved, proved.* If your search needs a rule you invented to finish a size, that rule is part of your claim: state it, prove it, and show the survivor list is unchanged when you switch it off. A size that only closes with an unproved rule is not certified; (v) *a second implementation, written independently of your first, that differs in method* — not the same algorithm twice — and the two survivor lists compared elementwise at every k you claim. Report any difference rather than reconciling it silently: a disagreement means one of them is wrong, and finding which is part of the cell.

Conjecture (Sun). If $k \geq 2$ residue classes $a_1 \pmod{m_1}, \dots, a_k \pmod{m_k}$ are pairwise disjoint, then $\gcd(m_i, m_j) \geq k$ for some $i \neq j$. We say the conjecture *holds at k* if this is true for that k .

1 Certified contiguous boundary

Definition 1. The *certified contiguous boundary* K^* is the largest K such that, for every $3 \leq k \leq K$:

1. both independent implementations (p4.codex, SAT-based; p4.dfs, column enumeration plus backtracking, no SAT) enumerated the survivor list at k , and the two lists agree elementwise after canonical normalisation;

2. every survivor is decided UNSAT together with its minimal part: the smallest UNSAT sub-multiset and explicit, directly checked residue witnesses for all its sub-multisets of one size less (p4_codex minimal-part certificate, §2);
3. the survivor lists, the decisions and the minimal-part files are attached (`code/data/`);
4. a timed rerun on this laptop takes under 10 minutes: the p4_codex pipeline (enumeration, SAT decisions, minimal parts) plus the p4_dfs enumeration, added together.

The list comparison comes from `p4_crosscheck/compare_report.json`, produced by `compare_lists.py`. The attached data (`code/data/`) are: every p4_codex `results_kK.json` for $k \leq 16$ (gzipped for $k \geq 13$), which contain the survivor lists, decisions and minimal-part records; every `minimal_parts_kK.jsonl.gz` and `minimal_witnesses_kK.jsonl.gz`; and the p4_dfs survivor and decision files for $k \leq 15$ (`code/data/p4_dfs/`, for $k = 15$ the merged 24-block decisions). The p4_dfs survivor list for $k = 16$ is not attached.

k	survivors (p4_dfs)	survivors (p4_codex)	elementwise	all UNSAT (both)
3–6	0	0	identical	vacuous
7	2	2	identical	yes
8	7	7	identical	yes
9	6	6	identical	yes
10	3	3	identical	yes
11	137	137	identical	yes
12	522	522	identical	yes
13	4395	4395	identical	yes
14	19 880	19 880	identical	yes
15	155 890	155 890	identical	yes
16	236 333	236 333	identical	p4_codex: yes; p4_dfs: partial

“Identical” means that, after canonical normalisation, the two lists coincide element by element: nothing is only in one list, there are no duplicates, and the SHA-256 values are equal (Section 4). No survivor received different decisions from the two implementations.

Besides p4_codex, p4_dfs decided every survivor UNSAT for every $k \leq 15$ (for $k = 15$ in 24 round-robin blocks, merged and checked). For $k = 16$, p4_codex decided all 236 333 survivors UNSAT; the p4_dfs decisions are a **partial cross-check**: 17/24 blocks, covering 167 404 survivors, all UNSAT and in agreement, after which the run was stopped (2026-09-26 15:52).

Timed rerun. Run on this laptop on 26 September 2026 (2026-09-26 16:04), with the code as submitted, in a copy of each directory, logs in `code/p4_crosscheck/timed_rerun/logs/`. Times are wall clock in seconds; p4_codex enumerates $k = 15$ with 6 worker processes, as in its original run; everything else is single-process.

k	p4_codex enum. + SAT	p4_codex minimal parts	p4_dfs enum.	total	< 600 s
3–12	3.2 (all sizes together)		see below		yes
13	6.9	11.1	2.0	20.0	yes
14	51.3	129.0	12.4	192.8	yes
15	187.1	286.9	139.5	613.5	no

The full p4_dfs run (enumeration *and* decisions) for $k = 3, \dots, 13$ took 82.7 s. In every rerun, all counts, survivor lists and SHA-256 values reproduced the original ones exactly: p4_dfs for $k = 3, \dots, 13$ (full run) and $k = 14, 15$ (enumeration). The p4_dfs decisions for $k = 14$ (971 s in the original run) were not rerun, and $k = 16$ was not rerun.

The boundary. Items 1–3 hold for every $3 \leq k \leq 15$; at $k = 16$ item 3 fails (the p4_dfs list for $k = 16$ is not attached) and no timed rerun was made. Item 4 holds for every $k \leq 14$ (table above). Hence

$$K^* = 14.$$

The sizes $k = 15, 16$ are *decided, beyond the certified boundary*: every survivor is UNSAT by p4_codex, with its minimal part in the attached files, and the two survivor lists are identical; they do not meet every condition of the definition.

2 Minimal-part certificates

For each UNSAT survivor, p4_codex/minimal_cert.py searches its sub-multisets by increasing size s with selector SAT: a Boolean y_i per class guards every pair’s disjointness clause, and assumptions activate exactly the selected classes. The first UNSAT sub-multiset (the *minimal part*) is recorded with its indices, moduli and gcd matrix. At size $s - 1$ every sub-multiset is SAT, and each distinct one has an explicit residue witness in minimal_witnesses_kK.jsonl.gz, checked directly over $\mathbb{Z}/L$ (bitset of occupied residues, no SAT involved). Checking size $s - 1$ suffices: every subfamily of a SAT family is SAT, so a smaller UNSAT part would extend to an UNSAT part of size $s - 1$. verify_minimal.py repeats the coverage and the direct check and re-solves every distinct core (log p4_codex/work/final_minimal_audit.log):

k	minimal parts	distinct cores	witnesses checked	size distribution s :count
7	2	2	10	5:2
8	7	6	54	3:2, 5:5
9	6	5	65	3:1, 5:5
10	3	3	73	5:3
11	137	30	454	3:84, 5:1, 6:1, 7:51
12	522	73	1 344	3:380, 4:10, 7:132
13	4 395	599	18 060	3:2178, 4:334, 5:363, 6:669, 7:547, 8:193, 9:108, 10:1, 11:2
14	19 880	1 716	76 108	3:11007, 4:2405, 5:1325, 6:2369, 7:1842, 8:638, 9:294
15	155 890	2 664	164 041	3:132232, 4:5478, 5:2030, 6:9153, 7:3007, 8:872, 9:3113, 10:1, 12:4
16	236 333	4 995	401 312	3:153452, 4:19118, 5:16279, 6:25914, 7:8557, 8:7396, 9:4190, 10:834, 11:581, 12:12

For $k \leq 6$ there are no survivors. Every minimal-part record, core and witness file is in code/data/p4_codex/; each survivor’s record in results_kK.json names its witness file and its SHA-256.

3 Vacuity demonstration

The same code, run with the gcd bound $[2, k]$ in place of $[2, k - 1]$ (`--bound-offset 0`), must find disjoint systems. It does. For $k = 3, 4, 5, 6$ the survivors are listed below, and the witness $r \bmod k$ ($0 \leq r < k$) is found and passes the literal (element-by-element) disjointness check:

k	survivors with bound $[2, k]$	SAT
3	(3, 3, 3)	(3, 3, 3)
4	(4, 4, 4, 4)	(4, 4, 4, 4)
5	(5, 5, 5, 5, 5)	(5, 5, 5, 5, 5)
6	4 survivors	(6, 6, 6, 6, 6, 6) only

The `p4.codex` vacuity runs (`results_kK_vacuity.json`, bound $[2, k]$ throughout, including $L = \text{lcm}(1, \dots, k)$) give the same picture.

- For $k = 3, 4, 5$ there is exactly one survivor, $(k, \dots, k)$. It is SAT, with witnesses $(0, 1, 2)$, $(0, 3, 2, 1)$ and $(0, 1, 2, 3, 4)$.
- For $k = 6$ there are 4 survivors:

$$(4, 6, 6, 6, 6, 12), \quad (6, 6, 6, 6, 6, 6), \quad (6, 6, 6, 6, 10, 15), \quad (6, 6, 6, 12, 15, 20).$$

Only $(6, 6, 6, 6, 6, 6)$ is SAT, with witness $(0, 2, 1, 3, 5, 4)$; the other three are UNSAT.

Every witness was checked directly over $\mathbb{Z}/L$ (`verified_over_Z_mod_L`). The survivor counts and the unique SAT family agree with `p4_dfs`.

4 Survivor lists

The canonical survivor lists are in the attached result files: `p4_dfs/results/kXX_sun.json` (for $k = 15, 16$, `p4_crosscheck/dfs_view/kXX_sun.json`) and `p4_codex/results_kXX.json`. After the canonical normalisation of `compare_lists.py`, both implementations give the following SHA-256 values, identical for every k :

k	survivors	canonical SHA-256
3–6	0	4f53cda18c2baa0c0354bb5f9a3ecbe5 ed12ab4d8e11ba873c2f11161202b945
7	2	01557c6ee033362572d9b27c7544c573 ac3114f221e0989214da4c72908f71da
8	7	1621baadfc46fc4bd51a7a75c6d635d1 98bca530ae676a0cec92b471313d8a77
9	6	febd71b7bbda2883058372c14ae266f5 66ba8c815d0005df1081c944df13144c
10	3	b02fcc22779027561f7c60a1d3ffba72 30f1b11d39e3617eb7361ffe883db76e
11	137	dee521791e4d6f13cbeaa1875c70c99e 7278c8820143c0ab7fc0fe31003723fe
12	522	d7ab51816f731ab2283f025ba0f31adf f2efe79f0762d2ec03fc01ed2d30986d
13	4 395	2dbfd65f26961615f44ddd6efced8fdf 1d8489920b9de2bcfa111ab6b7411dd1
14	19 880	f2ec65da7dedb65caec60c1acd7de28f c3bcb28a743a274dd240f32a038ed818
15	155 890	8a4aca6d7ee2df8bbc4f278621f82167 46990cf7c09164b452026bc0a2d87b40
16	236 333	2f8472ca4310cce90f832161c0152085 3a5c7b79516202f8d15ede27b81334fd

(The value for $k = 3$ –6 is the hash of the empty list.)

5 Toggle-off identity

Every pruning rule of `p4_dfs` can be switched off:

- `--no-gcd-prune`;

- `--no-coprime-prune`;
- `--late-nf`;
- `--no-interchange` and `--plain-decide`, for the decision step.

With pruning off, the output is identical:

- *Enumeration toggles*: identical survivor lists and stage counts for $3 \leq k \leq 7$, over all 2^3 combinations (the combinations with `--no-gcd-prune` only for $k \leq 6$).
- *Naive oracle*: agrees with a naive enumeration of all multisets for $k \leq 8$.
- *Deciders*: the three deciders agree on every survivor for $k \leq 10$. The two forward-checking deciders also agree for $k = 11, 12, 13$.

For `p4.codex`, the density-prefix pruning (its only shortcut) was switched off by the script `validate_pruning.py`, with the option

`--no-density-prefix-prune`.

For every $k = 3, \dots, 12$ the survivor lists and all filter counts are identical to the pruned run: MATCH at every size (`work/density_prune_comparison.json`).

6 Minimal counterexamples cannot have $k = 24$ or $k = 30$

This section is a complete proof by hand. The central injectivity idea (the map ω below) is taken from O'Bryant's proof of Lemma 6(8) in *On Z.-W. Sun's disjoint congruence classes conjecture* (2007). The write-up below is self-contained, and it needs only the hypothesis at $k - 1$.

Theorem 2. *Let $k \in \{24, 30\}$. If the conjecture holds at $k - 1$, then it holds at k . In particular, if the conjecture fails for some k , the least such k is neither 24 nor 30.*

Tools

Lemma 3. *$a \bmod m$ and $b \bmod n$ are disjoint if and only if $\gcd(m, n) \nmid a - b$.*

Proof. If x lies in both, then $a \equiv x \equiv b \pmod{\gcd(m, n)}$. Conversely, suppose $g = \gcd(m, n)$ divides $a - b = gt$, and take $um + vn = g$. Then $x = a - umt = b + vnt$ lies in both. $\square$

Fix a disjoint family $a_i \bmod m_i$ ($i \in [k]$), write $g_{ij} = \gcd(m_i, m_j)$, and let $\text{pr}(n)$ be the set of primes dividing n . By Lemma 3, every $g_{ij} \geq 2$ and $a_i \not\equiv a_j \pmod{g_{ij}}$. Let $\pi(i) = \{q \text{ prime} : q \mid g_{ij} \text{ for some } j \neq i\}$.

Lemma 4. *$\text{pr}(g_{ij}) = \pi(i) \cap \pi(j)$ for $i \neq j$. In particular, $\pi(i) \cap \pi(j) \neq \emptyset$.*

Proof. The inclusion $\subseteq$ is clear. If $q \in \pi(i) \cap \pi(j)$, then q divides m_i and m_j , hence g_{ij} . Finally $g_{ij} \geq 2$. $\square$

Proof of Theorem 2

Let $p = k - 1$, so $p = 23$ or $p = 29$, a prime. Suppose, for contradiction, that the family above has all $g_{ij} \leq p$. Let r be the number of primes less than p : $r = 8$ for $p = 23$ (the primes $2, \dots, 19$) and $r = 9$ for $p = 29$ (the primes $2, \dots, 23$).

Step 1: the set S . Let $S = \{i : p \in \pi(i)\}$. For $i \neq j$ in S , $p \mid g_{ij} \leq p$ by Lemma 4, so $g_{ij} = p$ and $\pi(i) \cap \pi(j) = \{p\}$. Conversely, if $g_{ij} = p$ then $i, j \in S$. Thus the pairs with gcd exactly p are exactly the pairs inside S .

Step 2: $|S| \geq 3$. Let $x \in [k]$. The family without x consists of $k - 1$ pairwise disjoint classes. Since the conjecture holds at $k - 1$, it contains a pair with $\gcd \geq k - 1 = p$. As all gcds are $\leq p$, that pair has gcd exactly p , so it lies inside $S \setminus \{x\}$. Hence $|S \setminus \{x\}| \geq 2$ for every x , and therefore $|S| \geq 3$. Write $s = |S|$.

Step 3: $S \neq [k]$. Otherwise every $g_{ij} = p$, so the a_i are pairwise incongruent modulo p . That forces $k \leq p = k - 1$, which is impossible. Let $O = [k] \setminus S$; then $|O| = k - s \geq 1$.

Step 4: the sets P_i . For $i \in S$ let $P_i = \pi(i) \setminus \{p\}$. By Step 1 the sets P_i ($i \in S$) are pairwise disjoint. Every prime in $\pi(i)$ divides some $g_{ij} \leq p$, and $p \notin P_i$. So P_i consists of primes less than p , and

$$\sum_{i \in S} |P_i| \leq r. \quad (1)$$

Let $l \in O$ and $i \in S$. By Lemma 4, $\pi(l) \cap \pi(i) = \text{pr}(g_{li})$ is nonempty, and it does not contain p because $l \notin S$. So $\pi(l) \cap P_i \neq \emptyset$; in particular every P_i is nonempty, and $s \leq r$.

Step 5: an injective map. Number $S = \{i_1, \dots, i_s\}$. For $l \in O$ put

$$\omega(l) = (\min(\pi(l) \cap P_{i_1}), \dots, \min(\pi(l) \cap P_{i_s})) \in P_{i_1} \times \dots \times P_{i_s}.$$

Suppose $\omega(l) = \omega(l')$ for some $l \neq l'$ in O . Each coordinate q lies in $\pi(l) \cap \pi(l')$, so $q \mid g_{ll'}$ by Lemma 4. The $s \geq 3$ coordinates are distinct primes, because the P_i are disjoint. Hence $g_{ll'} \geq 2 \cdot 3 \cdot 5 = 30 > p$, a contradiction. So ω is injective, and

$$k - s = |O| \leq \prod_{i \in S} |P_i|. \quad (2)$$

Step 6: counting. By AM-GM and (1), $\prod_{i \in S} |P_i| \leq \left(\frac{1}{s} \sum |P_i|\right)^s \leq (r/s)^s$, for $3 \leq s \leq r$. Numerically:

$p = 23, r = 8: s$	3	4	5	6	7	8	
$(8/s)^s$	18.97	16	10.49	5.62	2.55	1	
$k - s = 24 - s$	21	20	19	18	17	16	
$p = 29, r = 9: s$	3	4	5	6	7	8	9
$(9/s)^s$	27	25.63	18.90	11.39	5.81	2.57	1
$k - s = 30 - s$	27	26	25	24	23	22	21

In every column except ($p = 29, s = 3$) we have $(r/s)^s < k - s$, which contradicts (2). Each entry is easily checked by hand. For example, $(8/3)^3 = 512/27 < 19$, $(9/4)^4 = 6561/256 < 26$ and $(9/5)^5 = 59049/3125 < 19$.

Step 7: the equality case $p = 29, s = 3$. Equality in AM-GM, together with (1) and (2), forces the following:

$$\bullet \quad |P_{i_1}| = |P_{i_2}| = |P_{i_3}| = 3;$$

- the three sets partition all nine primes $2, 3, 5, \dots, 23$;
- $|O| = 27$, so ω is a bijection onto $P_{i_1} \times P_{i_2} \times P_{i_3}$.

Otherwise the product is an integer below 27, which again contradicts (2). Say $23 \in P_{i_a}$. Pick $b \neq a$ and any $q \in P_{i_b}$, and let c be the third index. There are exactly $|P_{i_c}| = 3$ triples in the product whose a -coordinate is 23 and whose b -coordinate is q . By bijectivity, these are $\omega(l)$ for three distinct $l \in O$. Take two of them, $l \neq l'$. Then 23 and q both divide $g_{ll'}$, so $g_{ll'} \geq 23q \geq 46 > 29$, a contradiction.

All cases are contradictory, which proves Theorem 2. $\square$

Remark 5. 1. The argument uses the hypothesis only at size $k - 1$, not at all smaller sizes.

2. This agrees with the second sentence of O'Bryant's Theorem 3 ("a counterexample to the DCCC with minimal k does not have $k \in \{24, 30\}$ "). Here it is re-proved, not assumed.
3. The same argument works for every $k \leq 30$ with $k - 1$ prime. An exact enumeration of all size vectors ($|P_i|$) (`p4_dfs/check_minimality.py`, not part of the proof) confirms that the strict inequality holds for $k \in \{4, 6, 8, 12, 14, 18, 20, 24\}$. The only equality case in the range is $k = 30$, $s = 3$, $(3, 3, 3)$, treated in Step 7. For $k - 1 \geq 31$ Step 5 breaks down, since $2 \cdot 3 \cdot 5 \leq k - 1$.